EXECUTABLE ROADMAP

APEX TERMINAL V9 REALITY

Structured for high-velocity execution coverage. Rigorous Definition of Done.
Strict separation of Engineering Reality and R&D Vision.

EXECUTION TRACK
Years 1-2

VISION TRACK
Years 3-4

EXECUTION TRACK

YEARS 1 & 2 // FOUNDATION & SCALE

WAVE 1 // WEEKS 1-13

DATA SPINE & ORIGINS

PRIMARY OBJECTIVES

- Establish canonical data model
- Deterministic Market Replay
- Telemetry Foundation

KEY DELIVERABLES

- Market Data Service
- Replay Engine
- Telemetry Logger

DEFINITION OF DONE // REALITY CHECK

- [x] Telemetry Emitted & Logged
- [x] Search Index Updated
- [x] Platform Health Monitor Green
- [x] Export Artifact Bundle Generated
- [x] Playwright MCP Headed Proof
- [ ] Video Walkthrough Recorded

01 Canonical Symbol Model & Resolver

Data Identity Layer

SPECS

- Data Model: SymbolRef(exchange: str, ticker: str, asset_class: str, figi: Optional[str], display_name: str). Store in SQLite symbols table with a UNIQUE constraint on (exchange, ticker).
- Resolver Logic: Exact-match first → prefix match → fuzzy match fallback. Cache resolved symbols in an LRU cache (maxsize=2048) to avoid repeated DB hits.
- Migration: Write an Alembic migration to create the symbols table and backfill from existing mock CSV headers and Finnhub reference data.

OUTPUTS

- SymbolRef Pydantic model in phase1/services/api/models/symbol.py
- Resolver service in phase1/services/symbol_resolver.py
- API endpoint in phase1/services/api/routes/symbols.py
- Alembic migration script
- ≥15 unit tests, ≥3 integration tests, ≥1 Playwright E2E test

02

Watchlist Panel & CRUD APIs

Core User Workflow

SPECS

- Backend: Two new tables – watchlists(id, name, user_id, created_at, updated_at) and watchlist_items(id, watchlist_id, symbol_ref_id, sort_order, added_at). Full REST API under /api/watchlists/.
- Frontend: React component WatchlistPanel with columns: Symbol, Last Price, Change %, Volume. Real-time updates via existing WebSocket price feed. Drag-to-reorder items using @dnd-kit/sortable.
- State Management: Zustand store watchlistStore with actions: addItem, removeItem, reorderItems, createWatchlist, deleteWatchlist.

OUTPUTS

- Watchlist SQLAlchemy models and Alembic migration
- CRUD API endpoints under /api/watchlists/
- WatchlistPanel React component with drag-to-reorder
- Zustand watchlistStore
- ≥12 API tests, ≥8 frontend tests, ≥2 E2E tests

03 Multi-Watchlist Support & Quick Actions

Workflow Acceleration

SPECS

- UI: Horizontal tab bar above the watchlist panel. Each tab = one watchlist. '+' button to create new. Right-click context menu for rename/delete.
- Quick Actions: Hover over a watchlist item reveals an action bar with icons: 📈 (open chart), 📊 (open analytics), ⏪ (run backtest). Each action dispatches a navigation event to the appropriate workspace.
- Keyboard shortcuts: Ctrl+1-9 to switch watchlists, Enter on selected item to open chart, Delete to remove from list.

OUTPUTS

- WatchlistTabs component with tab management
- QuickActionBar component with navigation integration
- Keyboard shortcut handlers registered in the command palette
- ≥5 Playwright E2E tests, ≥10 Vitest unit tests

04 Deterministic Alerts Engine & Alerts Board

Event-Driven Monitoring

SPECS

- Alert Types: PriceAbove, PriceBelow, VolumeSpike(threshold_multiplier), IndicatorCross(indicator, direction). Each alert has a condition evaluator function.
- Engine: AlertEngine class runs on each new bar/tick. Maintains an active_alerts dict. When a condition transitions from False→True, emit an AlertEvent to the event bus. Deterministic – same input always produces same alerts.
- Storage: alerts table (id, symbol_ref_id, alert_type, params_json, status, triggered_at, created_at). API: GET/POST/DELETE /api/alerts/.
- UI: AlertsBoard with real-time feed of triggered alerts, filterable by symbol/type/severity/time-range. Each alert row shows: time, symbol, condition, current value, status badge.

OUTPUTS

- AlertEngine service with condition evaluators
- Alerts REST API endpoints
- AlertsBoard React component
- Alembic migration for alerts table
- ≥20 unit tests, ≥2 determinism tests, ≥2 E2E tests

05

Watchlist-Alert Integration & Triggers

Workflow Automation

SPECS

- Watchlist Integration: Right-click a watchlist item → 'Set Alert' opens an alert configuration modal pre-populated with the symbol. Alert appears as a bell icon badge on the watchlist item.
- Trigger System: alert_triggers table (id, alert_id, action_type, action_params_json). Action types: NOTIFY_UI, NOTIFY_SOUND, PAPER_TRADE(side, qty). Triggers execute sequentially when an alert fires.
- Notification Service: In-app notification toast + optional sound. Notifications persisted in notifications table for history.

OUTPUTS

- Alert configuration modal component
- TriggerEngine service with action executors
- NotificationService with toast UI and sound support
- ≥10 integration tests, ≥3 E2E tests

06

Symbol Resolver UI Refinement

User Experience Polish

SPECS

- Disambiguation Modal: When resolver returns >1 match, show a modal listing all matches with exchange, asset class, and description. User clicks to select.
- Autocomplete: Debounced input (300ms) → call /api/symbols/search?q=... → display dropdown with up to 10 results. Highlight matching characters. Arrow keys to navigate, Enter to select.
- Accessibility: All interactive elements have aria-labels. Modal traps focus. Escape closes modal. Screen-reader announces result count.

OUTPUTS

- SymbolAutocomplete component with debounced search
- DisambiguationModal component
- Accessibility improvements across symbol search flow
- ≥8 Vitest tests, ≥3 Playwright E2E tests, axe-core accessibility audit

07

Centralized Event Log Service

Observability & Audit

SPECS

- Event Schema: events table (id, timestamp, event_type, actor, action, details_json, severity, correlation_id). event_type enum: ORDER, TRADE, ALERT, ERROR, CONFIG_CHANGE, LOGIN, SYSTEM.
- Service: EventLogger class with methods log_event(type, actor, action, details). Thread-safe with a write-ahead buffer flushed every 100ms or 50 events.
- API: GET /api/events/ with pagination, filters (type, severity, actor, time_range). Deterministic ordering by (timestamp, id).
- Integration: Instrument existing order execution, alert triggering, and error handling to emit events via EventLogger.

OUTPUTS

- EventLogger service with write-ahead buffer
- Events table Alembic migration
- Events REST API with pagination and filtering
- Instrumentation hooks in order/alert/error paths
- ≥15 unit tests, ≥2 concurrency tests

08 Event Log UI Panel

Observability & Audit

SPECS

- Component: EventLogPanel with a professional data table. Columns: Timestamp, Type (with colored badge), Actor, Action, Severity (icon), Details (expandable).
- Filters: Sidebar filter panel with checkboxes for event types, severity dropdown, date range picker. Filters compose as AND conditions on the API query.
- Real-time: New events stream via WebSocket and prepend to the table. Visual flash animation on new rows.
- Export: 'Export CSV' button downloads filtered events as a CSV file.

OUTPUTS

- EventLogPanel component with filter sidebar
- Real-time event streaming via WebSocket
- CSV export functionality
- ≥8 Vitest tests, ≥3 integration tests, ≥2 E2E tests

09

Time-Series Event Replay API

Deterministic Debugging

SPECS

- API: GET /api/events/replay?start=...&end=...&speed=1x returns a streaming response of events in chronological order.
- Playback Engine: EventReplayEngine class that reads events from DB and emits them at configurable speed (0.5x, 1x, 2x, 5x, 10x). Integrates with existing replay infrastructure.
- UI: Timeline scrubber component below the event log panel. Play/Pause/Stop buttons. Speed selector dropdown. Current timestamp indicator. Scrubber can be dragged to jump to any point.
- Visual: During replay, the Event Log panel shows events appearing in real-time as they 'replay'. Other panels (chart, portfolio) optionally sync to replay timestamp.

OUTPUTS

- EventReplayEngine service
- Streaming replay API endpoint
- TimelineScrubber React component
- Replay controls integration with Event Log panel
- ≥10 unit tests, ≥3 E2E tests

16 Scenario-Driven Alert Templates

User Productivity

SPECS

- Template Schema: alert_templates table (id, name, description, category, conditions_json, default_params). Categories: Breakout, Reversal, Volume, Momentum, Custom.
- Built-in Templates: Price Breakout (price crosses above N-day high), Volume Surge (volume > 3× 20-day average), RSI Overbought/Oversold, MACD Crossover, Bollinger Band Squeeze.
- UI: Template Gallery accessible from Alerts workspace and from watchlist context menu. Each template shows a description, preview of conditions, and 'Apply' button that pre-fills the alert creation form.
- Smart Suggestions: When adding a symbol to a watchlist, show a tooltip: 'Set up a Breakout Alert?' based on recent price action.

OUTPUTS

- Alert template data model and seed data (≥8 built-in templates)
- AlertTemplateGallery component
- Smart suggestion logic in watchlist integration
- ≥15 unit tests, ≥3 E2E tests

11 Multi-Workspace Layout Engine

Personalization Infrastructure

SPECS

- Layout Engine: Each workspace is a serializable JSON object: { id, name, panels: [{ type, position: {x,y,w,h}, config }] }. Uses react-grid-layout for drag/resize.
- Storage: workspaces table (id, user_id, name, layout_json, is_default, created_at, updated_at). API: GET/POST/PUT/DELETE /api/workspaces/.
- Persistence: On every panel move/resize, debounce 2s then auto-save layout to backend. On load, fetch the user's active workspace layout and restore panel positions.
- Workspace Switcher: Dropdown in the app header showing all saved workspaces. Click to switch. Star icon to set default.

OUTPUTS

- WorkspaceLayoutEngine with react-grid-layout integration
- Workspaces REST API
- WorkspaceSwitcher component
- Auto-save with debounce logic
- ≥12 unit tests, ≥4 E2E tests, ≥8 API tests

12

Workspace Management UI Controls

Personalization Infrastructure

SPECS

- Workspace Menu: Gear icon in workspace switcher → dropdown with: Save, Save As, Reset to Default, Delete, Manage Workspaces.
- Save As: Modal dialog with name input and optional description. Creates a clone of the current layout.
- Reset to Default: Confirmation dialog → replaces current layout_json with the factory default. Default layouts defined in a seed configuration file.
- State Handoff: When switching workspaces, serialize transient state (selected symbol, scroll positions) into the outgoing workspace and restore from the incoming workspace.

OUTPUTS

- WorkspaceMenu component with all actions
- SaveAsModal component
- Factory default layout configuration
- State serialization/restoration logic
- ≥6 E2E tests, ≥5 unit tests

13

Profile-Based Layout Defaults & Q1 Retrospective

Onboarding & Review

SPECS

- Preset Profiles: 'Trader' (chart + watchlist + orders + positions), 'Researcher' (chart + backtest + strategy editor + analytics), 'Risk Manager' (dashboard + risk scenarios + portfolio + alerts). Stored as JSON templates.
- First-Run Experience: On first login, show a profile selection modal. User picks a profile → system clones the template as their default workspace.
- Q1 Audit: Run all tests (pytest + Vitest + Playwright). Generate coverage report. Document any tech debt accumulated. Update README with Q1 features.

OUTPUTS

- 3 preset workspace profile templates
- FirstRunProfileSelector component
- Q1 retrospective document with metrics
- Updated README.md with Q1 feature summary
- Full test suite pass report

EXECUTION TRACK // ACTIVE

WAVE 2 // WEEKS 14-26

AUTOMATION CORE

PRIMARY OBJECTIVES

- Trigger System
- Search Indexing
- Performance Visualization

KEY DELIVERABLES

- Automation Engine
- Global Search
- Analytics Dashboard

DEFINITION OF DONE // REALITY CHECK

- [x] Telemetry Emitted & Logged
- [x] Search Index Updated
- [x] Platform Health Monitor Green
- [x] Export Artifact Bundle Generated
- [x] Playwright MCP Headed Proof
- [ ] Video Walkthrough Recorded

14 Dynamic LeftNav Redesign

Navigation Architecture

SPECS

- Collapsible section groups: Main (Dashboard, Trading, Portfolio, Orders), Tools (Backtest, Strategy, Alerts, Replay), System (Ops, Settings, Audit).
- Active workspace indicator with animated highlight. Tooltip on hover showing workspace name.
- Responsive: collapses to icon-only on narrow screens, expands on hover.

OUTPUTS

- Refactored LeftNav component with section grouping
- Responsive behavior (collapse to icons)
- ≥10 Vitest tests, ≥5 E2E tests, accessibility audit

15 Personalized Dashboard Pinning

Personalization

SPECS

- Pin API: POST /api/dashboard/pins {component_type, config_json, position}. Pinned items stored in user_pins table.
- Pin Button: Small 📌 icon on every pinnable component header. Click to toggle pin.
- Pin Manager: Grid view of all pinned items with drag-to-rearrange. Remove button on each.

OUTPUTS

- PinButton component
- PinManager dashboard view
- user_pins table and migration
- Pin REST API
- ≥6 E2E tests, ≥8 API tests

16

Expanded Search Index (Strategies, Backtests, Artifacts)

Search Infrastructure

SPECS

- Search Index: FTS5 virtual table in SQLite: search_index(id, object_type, title, description, metadata_json, updated_at).
- Indexer: Background job that watches for new/updated strategies/backtests and upserts into the search index.
- API: GET /api/search?q=...&types=symbol,strategy,backtest with faceted results grouped by type.
- Result Ranking: Symbols weighted 2x, recently-modified items boosted by recency factor.

OUTPUTS

- FTS5 search index schema and migration
- Background indexer service
- Expanded search API with type filtering
- Updated search results UI with type badges
- ≥15 unit tests, ≥5 integration tests

17 Search Results Deep-Linking

Search UX

SPECS

- Result Action Map: { symbol: navigate('/ui2/trading', {symbol}), strategy: navigate('/ui2/backtest', {strategyId}), backtest: navigate('/ui2/runs', {runId}), alert: navigate('/ui2/alerts', {alertId}) }.
- URL State: Each workspace reads query params on mount to pre-select the referenced item. e.g., /ui2/trading?symbol=AAPL.
- History: Search results maintain browser history so Back button returns to search results.

OUTPUTS

- Deep-link navigation handlers for all result types
- URL query parameter support in each workspace
- Browser history integration
- ≥8 E2E tests, ≥5 Vitest tests

18

Full-Text Search on Telemetry & Logs

Observability

SPECS

- Extend FTS5 index to include event log entries: search_events_fts(event_id, message_text, details_text).
- API: GET /api/events/search?q=...&severity=...&start=...&end=... with highlighted text snippets in results.
- Performance: Index only last 30 days of events. Older events searchable via archive query (slower).
- UI: Search bar within Event Log panel with instant results. Matching text highlighted in yellow.

OUTPUTS

- Events FTS5 index extension
- Event search API with text highlighting
- In-panel search bar for Event Log
- ≥10 unit tests, ≥2 performance tests, ≥2 E2E tests

19

Saved Searches & Search History

User Productivity

SPECS

- Storage: saved_searches table (id, user_id, name, query, filters_json, created_at). API: CRUD under /api/searches/.
- Search History: Last 20 searches stored in localStorage. Shown in a dropdown below the search bar when focused with no input.
- Saved Search Chips: Pinned saved searches appear as clickable chips above search results for one-click re-execution.
- Management: Modal accessible from search dropdown → list all saved searches → rename/delete actions.

OUTPUTS

- saved_searches table and migration
- Saved search CRUD API
- SearchHistoryDropdown component
- SavedSearchManager modal
- ≥8 API tests, ≥4 E2E tests, ≥5 Vitest tests

20

Search Relevance & Ranking Enhancements

Search Quality

SPECS

- Ranking Algorithm: score = tf_idf_score * recency_boost * popularity_factor. Recency: items modified in last 7 days get 1.5x boost. Popularity: items accessed more frequently get up to 1.3x boost.
- Synonyms: Synonym dictionary (e.g., 'USA' ↔ 'United States', 'MA' ↔ 'Moving Average'). Applied at query time to expand search terms.
- Stop Words: Common words ('the', 'a', 'an', 'is', 'in') are filtered from queries to improve precision.
- A/B Metrics: Log search result clicks with position. Calculate MRR (Mean Reciprocal Rank) to measure quality.

OUTPUTS

- Enhanced ranking algorithm with recency/popularity boosts
- Synonym dictionary (~50 entries)
- Stop-word filter list
- Search quality metrics logging
- ≥12 unit tests, ≥3 E2E tests

21 Virtual Scrolling for Large Data Tables

Performance Optimization

SPECS

- Library: @tanstack/react-virtual for virtualization. Apply to: Event Log, Orders, Trade History, Telemetry tables.
- Implementation: Measure row height (fixed 36px for data rows). Render only visible rows ± 10 row overscan buffer.
- Memory: Estimated max 10K rows in memory at any time. Older data fetched on scroll via infinite-scroll pagination.
- Visual: Smooth scrollbar. Row selection preserved during scroll. Sticky header always visible.

OUTPUTS

- VirtualizedTable component wrapping @tanstack/react-virtual
- Applied to Event Log, Orders, Trade History panels
- Infinite scroll pagination integration
- ≥5 performance tests, ≥8 functional tests

22

Backend Pagination & Deterministic Ordering

API Quality

SPECS

- Pagination Model: Cursor-based pagination using (timestamp, id) as cursor. Response: { data: [...], next_cursor: '...', has_more: bool }.
- Endpoints: /api/events/, /api/orders/, /api/trades/, /api/alerts/, /api/search/.
- Default page size: 50. Max: 200. Configurable via ?limit= query parameter.
- Ordering: Primary sort by timestamp DESC, secondary by id DESC. Consistent across pages.

OUTPUTS

- Cursor-based pagination utility class
- Updated all list endpoints with pagination
- Pagination response schema
- ≥20 API tests including edge cases

23

Chart Performance for Long Time Series

Rendering Performance

SPECS

- Downsampling: LTTB (Largest Triangle Three Buckets) algorithm. When viewport shows >2000 bars, downsample to 2000 representative points.
- Implementation: Backend endpoint /api/bars?symbol=...&tf=1D&start=...&end=...&max_points=2000. Server-side LTTB before response.
- Caching: Cache downsampled data keyed by (symbol, tf, start, end, max_points). Invalidate when new bars arrive.
- GPU Acceleration: Ensure Lightweight Charts uses WebGL rendering mode for >5000 points.

OUTPUTS

- LTTB downsampling utility (Python backend)
- Updated bars endpoint with max_points parameter
- Downsampled data caching layer
- ≥5 performance tests, ≥3 visual comparison tests

24 Chunked Data Loading for Backtests

Data Pipeline

SPECS

- Backend: StreamingResponse that yields JSON chunks of 100 results each. Client reads chunks via ReadableStream API.
- Progress: Backend sends total count in first chunk header. Frontend calculates and displays progress bar.
- UI: Results table renders rows as chunks arrive. Spinner at bottom while loading continues.
- Cancellation: User can cancel ongoing load. Backend respects connection close.

OUTPUTS

- StreamingResponse helpers for chunked JSON
- Frontend ReadableStream consumer utility
- ProgressBar component for data loading
- Cancel support in data loading pipeline
- ≥8 integration tests, ≥4 E2E tests

25

Unified Caching Layer

Architecture

SPECS

- CacheManager: Singleton class with get(key), set(key, value, ttl), invalidate(key), and invalidatePattern(pattern).
- Tiers: L1 = in-memory Map (fast, 100MB max), L2 = localStorage (persistent, 5MB max per entry). On miss: L1 → L2 → network.
- Key Naming: 'bars:{symbol}:{tf}:{range}', 'search:{query}:{type}', 'workspace:{id}:layout'.
- Cache Invalidation: WebSocket events trigger invalidation. e.g., new bar → invalidate 'bars:{symbol}:*'.

OUTPUTS

- CacheManager singleton with L1/L2 tiers
- Cache key naming convention documentation
- WebSocket-driven cache invalidation
- Instrumentation metrics (hit rate, miss rate)
- ≥15 unit tests, ≥5 integration tests

26 Compliance & Audit Bundle V1

Compliance & Q2 Review

SPECS

- Bundle Format: ZIP file containing JSON files for orders, positions, events, autopilot decisions, and a manifest.json with hashes.
- Hash Chain: Each JSON file includes a SHA-256 hash. manifest.json contains all file hashes plus a bundle hash (hash of all hashes).
- API: POST /api/compliance/export {start_date, end_date} → returns a downloadable ZIP.
- UI: Export button in the Ops workspace with date range picker. Download progress bar.
- Q2 Retrospective: Run full test suite, measure performance benchmarks, document improvements.

OUTPUTS

- Compliance export API endpoint
- BundleGenerator service with hash chain
- Export UI in Ops workspace
- Q2 retrospective document
- ≥10 integration tests, ≥2 E2E tests

WAVE 3 // WEEKS 27-39

AUTOPILOT & RISK

PRIMARY OBJECTIVES

- Risk Engine Integration
- Autopilot Logic
- Safety Rails

KEY DELIVERABLES

- Risk Monitor
- Autopilot V1
- Circuit Breakers

DEFINITION OF DONE // REALITY CHECK

- [x] Telemetry Emitted & Logged
- [x] Search Index Updated
- [x] Platform Health Monitor Green
- [x] Export Artifact Bundle Generated
- [x] Playwright MCP Headed Proof
- [ ] Video Walkthrough Recorded

27 Audit Bundle Export Integration

Compliance

SPECS

- Export Panel: Date range picker + 'Generate Bundle' button + download link. Progress bar during generation.
- Verification: Standalone Python script verify_bundle.py that re-hashes all files and compares to manifest.
- Signatures: Add optional HMAC signing with a server-side secret. Include HMAC in manifest for institutional users.

OUTPUTS

- Export panel UI in Ops workspace
- verify_bundle.py CLI tool
- HMAC signing support
- ≥5 E2E tests, ≥3 unit tests

28

Role-Based Access Control (RBAC)

Security

SPECS

- Roles: Admin (full access), Trader (trade, view, configure), Viewer (read-only). Stored in users.role column.
- Token: JWT with role claim. Middleware checks permissions on every request. Decorator @require_role('admin').
- Permission matrix: Define which endpoints each role can access. Store in a permissions configuration file.

OUTPUTS

- RBAC middleware and decorators
- Permission matrix configuration
- JWT authentication with role claims
- ≥15 API tests, ≥3 E2E tests

29

User Login/Logout & Session Management

Authentication

SPECS

- Login page: Username/password form → POST /api/auth/login → receive JWT. Store in httpOnly cookie.
- Session: JWT expires after 8 hours. Refresh token mechanism for seamless re-authentication.
- UI adaptation: Role stored in Zustand authStore. Components check role before rendering privileged actions.

OUTPUTS

- LoginPage component
- Auth API endpoints (login, logout, refresh)
- authStore with role-aware rendering
- Session expiry handling
- ≥8 E2E tests, ≥10 API tests

30

Audit Log UI Component

Governance

SPECS

- Audit events: LOGIN, LOGOUT, CONFIG_CHANGE, ORDER_PLACED, ALERT_CREATED, EXPORT_GENERATED, ROLE_CHANGED.
- Storage: audit_log table (id, user_id, action, details_json, ip_address, timestamp).
- UI: AuditLogPanel in the Ops workspace. Admin-only access. Filterable by user, action, date range.

OUTPUTS

- audit_log table and migration
- Audit logging middleware
- AuditLogPanel component (admin-only)
- ≥12 integration tests, ≥4 E2E tests

31 Plugin Interface Architecture

Extensibility

SPECS

- Plugin API: PluginBase class with hooks: on_init(), on_bar(bar), on_trade(trade), on_shutdown(). Plugins register via a manifest.json.
- Sandboxing: Plugins run in a subprocess with limited memory (256MB) and CPU (1 core). Communication via JSON-RPC over stdio.
- Registry: plugins table (id, name, version, enabled, manifest_json). API: GET/POST/DELETE /api/plugins/.

OUTPUTS

- PluginBase abstract class with lifecycle hooks
- Plugin sandboxing via subprocesses
- Plugin registry API
- Sample risk-check plugin
- ≥10 unit tests, ≥5 integration tests

32

Plugin Manager UI

Extensibility

SPECS

- UI: PluginManagerPanel with table: Name, Version, Status (Active/Inactive/Error), Actions (Enable/Disable/Remove).
- Install flow: Upload plugin ZIP → backend validates manifest → extracts to plugins/ directory → adds to registry.
- Safety: Only admin can manage plugins. Each action logged in audit log.

OUTPUTS

- PluginManagerPanel component
- Plugin upload/install API
- Plugin enable/disable lifecycle management
- ≥5 E2E tests, ≥5 API tests

33

Sample Plugin & Developer Docs

Ecosystem

SPECS

- Sample Plugin: risk_check_plugin that on_trade checks if position size exceeds a configurable threshold and emits a warning.
- Dev Sandbox: Standalone dev environment with mock data feed. Developers can test plugins locally without the full platform.
- Documentation: Plugin API reference, getting-started guide, template project, and example recipes.

OUTPUTS

- risk_check_plugin sample with tests
- Plugin developer sandbox setup script
- Plugin Development Guide (Markdown)
- Plugin API reference documentation
- ≥8 plugin unit tests

34

Plugin Telemetry Integration

Observability

SPECS

- Plugin events tagged with source='plugin:{name}' in the event log.
- Determinism contract: Plugins must not use randomness or wall-clock time. Platform injects a deterministic clock.
- Telemetry: Plugin execution time, memory usage, and error rate tracked per plugin.

OUTPUTS

- Plugin telemetry hooks
- Deterministic clock injection for plugins
- Plugin health metrics in Ops dashboard
- ≥8 unit tests, ≥3 integration tests

35

UI/UX Consistency Pass

Design Quality

SPECS

- Design Tokens: Consolidate all design values into CSS custom properties: --spacing-xs through --spacing-xl, --font-size-body, --color-primary, etc.
- Audit: Check all 13 workspaces for inconsistencies. Create a spreadsheet tracking issues.
- Fixes: Standardize button sizes, input field heights, table row heights, modal widths, and border radii.

OUTPUTS

- Consolidated design tokens in CSS custom properties
- UI consistency fix PRs for all 13 workspaces
- Design system specification document
- Visual regression test suite
- ≥13 visual regression tests (1 per workspace)

36

Multi-User Autopilot & Kill Switch

Automation

SPECS

- Portfolio isolation: Each Autopilot instance scoped to a user_id + portfolio_id. No cross-portfolio data access.
- Kill Switch: POST /api/autopilot/{user_id}/kill → immediately stops all Autopilot activity. Red 'KILL' button in Autopilot workspace.
- Concurrency: Multiple Autopilots run in separate threads. Verify thread isolation and no shared mutable state.

OUTPUTS

- Multi-user Autopilot manager
- Kill switch API and UI button
- Portfolio isolation enforcement
- ≥10 integration tests, ≥5 concurrency tests, ≥2 E2E tests

37

User Invitation & Approval Workflow

Team Features

SPECS

- Invitation: POST /api/users/invite {email, role} → generates invite token. Email (or display invite link in UI for demo).
- Signup: Invited user visits signup page with token → creates account → status set to 'pending'.
- Approval: Admin sees pending users in User Management panel → approve or reject. Approved users get assigned role.

OUTPUTS

- Invite API and token generation
- Signup page with invite token validation
- User Management admin panel
- ≥10 API tests, ≥5 E2E tests

38

Compliance Features Polish

Governance

SPECS

- Compliance Dashboard: Summary view showing: total orders, total events, active plugins, last export date.
- Export Summary: CSV/PDF of aggregated compliance metrics: daily order count, daily P&L, alert triggers, plugin activity.
- Data Retention: Configurable retention policy (default 1 year). Auto-archive old data.

OUTPUTS

- ComplianceDashboard component
- Summary export API (CSV + PDF)
- Data retention policy configuration and cleanup job
- ≥8 integration tests, ≥3 E2E tests

39

Cross-Currency Trading Support

Core Platform & Q3 Review

SPECS

- Currency Model: Portfolio has a base_currency. Non-base positions converted using FX rates.
- FX Rates: GET /api/fx/rates?base=USD returns current rates. Cached for 1 hour. Mock rates for demo.
- P&L: Unrealized P&L calculated in position currency then converted to base. Show both in UI.
- Q3 Review: Run all tests. Benchmark key paths. Document new tech debt. Plan Q4 priorities.

OUTPUTS

- FX rate service and API
- Currency-aware P&L calculations
- Currency selector in portfolio settings
- Q3 retrospective document
- ≥12 unit tests, ≥4 integration tests

WAVE 4 // WEEKS 40-52

ENTERPRISE SCALE

PRIMARY OBJECTIVES

- High-Frequency API
- Multi-Tenancy
- Audit Logging

KEY DELIVERABLES

- API Gateway
- Tenant Manager
- Compliance Reports

DEFINITION OF DONE // REALITY CHECK

- [x] Telemetry Emitted & Logged
- [x] Search Index Updated
- [x] Platform Health Monitor Green
- [x] Export Artifact Bundle Generated
- [x] Playwright MCP Headed Proof
- [ ] Video Walkthrough Recorded

40

Cross-Asset Exposure Engine

Risk Management

SPECS

- Exposure Model: Normalize all positions to notional USD value. Asset-specific risk weights: equities 1.0, options delta-adjusted, crypto 1.5x for volatility.
- Risk Report: Exposure by asset class, by sector, by individual position. Heatmap visualization.
- Limits: Configurable per-asset-class exposure limits. Breach triggers warning alert.

OUTPUTS

- Cross-asset ExposureEngine
- Risk exposure report API
- Exposure limits configuration
- Risk heatmap component
- ≥15 unit tests, ≥5 integration tests

41 Real-Time WebSocket Upgrade

Infrastructure

SPECS

- WebSocket Hub: Central WS connection manager. Single WS connection per client, multiplexed channels: price, alerts, events, orders.
- Protocol: JSON messages with {channel, action, data}. Heartbeat every 30s. Auto-reconnect with exponential backoff (1s, 2s, 4s, max 30s).
- Server: FastAPI WebSocket endpoint at /ws. Fan-out to subscribed channels. Server-side connection tracking.

OUTPUTS

- WebSocket Hub server implementation
- Client-side WS connection manager
- Channel multiplexing (price, alerts, events, orders)
- Heartbeat and reconnection logic
- ≥10 integration tests, ≥3 load tests

42 Advanced Charting Features

Analytics

SPECS

- Overlays: Up to 5 symbols on one chart. Each gets a separate Y-axis (right side). Legend with checkboxes to toggle.
- Mini-Map: Small overview chart below main chart showing full date range. Draggable viewport rectangle.
- Annotations: Click to add text, arrow, or shape annotations. Persisted in chart_annotations table. Shareable via URL.

OUTPUTS

- Multi-asset overlay system
- MiniMap component
- Annotation tools with persistence
- chart_annotations table and API
- ≥8 E2E tests, ≥10 unit tests

43

Data Ingestion Connectors

Data Layer

SPECS

- Connector Interface: DataConnector abstract class with methods: connect(), subscribe(symbol), get_bars(symbol, tf, range), disconnect().
- New Connectors: Implement at least one additional provider (e.g., Polygon.io or Alpha Vantage). Follow the existing Finnhub/Alpaca patterns.
- Provider Manager: Hot-swap providers without restart. Automatic failover if primary provider disconnects.

OUTPUTS

- DataConnector abstract class
- New provider connector implementation
- ProviderManager with hot-swap and failover
- ≥12 unit tests, ≥5 integration tests, ≥2 failover tests

44 Portfolio Backtester & Parameter Sweeps

Quantitative

SPECS

- Portfolio Backtest: Run N strategies on M assets. Aggregate P&L, drawdown, and Sharpe across the portfolio.
- Parameter Sweep: Define parameter grid (e.g., SMA periods [10,20,50,100]). Run backtest for each combination.
- Job System: Background job queue for backtests. Status polling via API. Concurrent execution with configurable parallelism.
- Results UI: Heatmap of parameter combinations colored by Sharpe ratio. Table with sortable metrics.

OUTPUTS

- PortfolioBacktester engine
- ParameterSweep runner
- Background job queue system
- Sweep results heatmap component
- ≥15 unit tests, ≥5 integration tests

45 Scheduled Backtests

Automation

SPECS

- Scheduler: scheduled_jobs table (id, cron_expression, job_type, params_json, next_run_at, last_run_at, status).
- Cron Engine: Parse cron expressions. Ticker checks every 60s for due jobs. Submits to job queue.
- UI: Schedule modal with frequency picker (daily, weekly, custom cron). Calendar view of upcoming scheduled jobs.

OUTPUTS

- CronScheduler engine
- scheduled_jobs table and migration
- Schedule management API
- ScheduleModal and ScheduleCalendar components
- ≥8 integration tests, ≥3 concurrency tests, ≥3 E2E tests

46

Performance Bottleneck Analysis

Optimization

SPECS

- Profiling: Use cProfile on key API endpoints. Identify top 10 slowest paths.
- WebSocket: Batch price updates (send every 100ms instead of per-tick). JSON → MessagePack for 60% size reduction.
- Database: Add covering indexes on frequently-queried columns. Optimize N+1 queries with eager loading.
- Frontend: React.memo on heavy components. useMemo for expensive computations.

OUTPUTS

- Performance profiling report with findings
- Optimized queries and indexes
- WebSocket batching and MessagePack
- React rendering optimizations
- Before/after benchmark results

47 Compliance Enhancements & Digital Signing

Enterprise

SPECS

- Digital Signing: Ed25519 key pair. Sign export bundles. Verification script updated to check signatures.
- Log Checksums: Each audit log entry includes hash of prev entry (hash chain). Any gap/modification is detectable.
- Export Enhancements: Include strategy source code snapshots and configuration state in compliance bundles.

OUTPUTS

- Ed25519 signing for exports
- Hash chain for audit log entries
- Enhanced export contents
- verify_signatures.py tool
- ≥10 integration tests, ≥3 tamper tests

48 Public Release Preparation

Release

SPECS

- Smoke Test Suite: 20 critical-path E2E tests covering: login → trade → backtest → autopilot → export.
- Bug Bash: Allocate 3 days for fixing bugs found during smoke testing. Prioritize by severity.
- Release Checklist: Version bump, changelog, README update, Docker image build, deployment docs.

OUTPUTS

- Smoke test suite (20 tests)
- Bug fixes for all critical/high issues
- Release checklist document
- Updated changelog and README
- Release candidate Docker image

49 Multi-Provider Data Expansion

Data Layer

SPECS

- Provider Priority: Ordered list of providers per asset class. e.g., equities: [Finnhub, Alpaca, Yahoo], crypto: [custom, Yahoo].
- Fallback Logic: If provider #1 returns no data, try #2 automatically. Log which provider served each request.
- Data Merging: For historical data, merge from multiple providers to fill gaps. Deduplicate by timestamp.

OUTPUTS

- ProviderPriority configuration system
- Automatic fallback routing
- Historical data merger
- Provider source tracking per bar
- ≥10 integration tests, ≥5 merger tests

50

Cross-Exchange Price Aggregation

Data Layer

SPECS

- Aggregator: PriceAggregator class that receives ticks from multiple providers and emits best bid/ask.
- Time Sync: Normalize all timestamps to UTC. Handle provider clock drift (max allowed: 5s).
- Deduplication: Hash of (symbol, timestamp, price, volume) to detect and drop duplicates.
- UI: Show consolidated price with a 'Sources' tooltip showing contributing providers.

OUTPUTS

- PriceAggregator service
- Time synchronization logic
- Deduplication filter
- UI source indicator tooltip
- ≥12 unit tests, ≥5 integration tests

51 Advanced Portfolio Risk Metrics

Quantitative

SPECS

- VaR Methods: Historical (sort returns, take percentile), Parametric (assume normal, use μ ± zσ). 95% and 99% confidence levels.
- CVaR: Expected loss beyond VaR threshold. More informative for tail risk.
- Metrics Dashboard: RiskMetricsPanel showing VaR, CVaR, Max Drawdown, Sharpe, Sortino, Calmar ratios.
- History: Store daily risk metrics for trend analysis. Chart risk metrics over time.

OUTPUTS

- RiskMetricsEngine with VaR, CVaR, drawdown, Sharpe calculations
- RiskMetricsPanel component
- Daily risk metrics storage and trend charts
- ≥20 unit tests, ≥5 integration tests, ≥3 E2E tests

52

Year 1 Retrospective & Year 2 Planning

Review & Planning

SPECS

- Final Test Run: All test suites – target 100% pass rate. Generate coverage report – target ≥80%.
- Metrics: Lines of code, test count, feature count, performance benchmarks, uptime stats.
- Retrospective: What went well, what didn't, what to improve. Document in a formal retrospective report.
- Year 2 Preview: Prioritize Year 2 themes based on user feedback and tech debt analysis.

OUTPUTS

- Year 1 Retrospective Report
- Final test results and coverage report
- Performance benchmark comparison (Q1 vs Q4)
- Year 2 Roadmap Draft
- v1.0 Release Notes

WAVE 5 // WEEKS 53-78

DISTRIBUTED SYSTEMS

PRIMARY OBJECTIVES

- Microservices Migration
- Kubernetes Orchestration
- Global State

KEY DELIVERABLES

- K8s Clusters
- Service Mesh
- Distributed DB

DEFINITION OF DONE // REALITY CHECK

- [x] Telemetry Emitted & Logged
- [x] Search Index Updated
- [x] Platform Health Monitor Green
- [x] Export Artifact Bundle Generated
- [x] Playwright MCP Headed Proof
- [ ] Video Walkthrough Recorded

53

Multi-Tenant Database Architecture

Architecture

SPECS

- Schema: Add tenant_id column to users, portfolios, watchlists, orders, etc.
- Middleware: TenantContextMiddleware extracts tenant_id from subdomain or header.
- Isolation: Use SQLAlchemy/Alembic mixin to automatically add tenant_id filter to all queries. Postgres RLS for strict enforcement if migrating to PostgreSQL.

OUTPUTS

- Multi-tenant schema migration
- TenantContext middleware
- Database query filters for isolation
- ≥20 unit tests, ≥5 integration tests

54

Tenant Management API & Admin UI

Admin Features

SPECS

- Models: tenants table (id, name, subdomain, plan_id, created_at). tenant_config table.
- Super Admin UI: Separate workspace for platform owners. List all tenants, view usage stats, toggle access.
- Provisioning: POST /api/admin/tenants creates schema/data and admin user for the new tenant.

OUTPUTS

- Tenant management API
- Super Admin workspace
- Tenant configuration system
- ≥10 API tests, ≥3 E2E tests

55

White-Labeling & Branding Engine

Customization

SPECS

- Theme Engine: Inject CSS variables from tenant_config at runtime. :root { --color-primary: var(--tenant-primary); }.
- Asset Storage: S3/MinIO bucket per tenant for logos/favicons.
- Routing: Middleware maps Host header (trading.acme.com) to tenant_id.

OUTPUTS

- Dynamic theming engine
- Logo/asset upload API
- Subdomain routing logic
- ≥5 visual tests, ≥5 API tests

56 SSO Integration (SAML/OIDC)

Enterprise Security

SPECS

- Library: Use python3-saml or authlib. Endpoints: /api/auth/sso/login/{tenant}, /api/auth/sso/callback.
- Config: Store IdP metadata xml or client_id/secret in tenant_config (encrypted).
- JIT Provisioning: Auto-create user on first successful SSO login if they don't exist.

OUTPUTS

- SAML/OIDC authentication backend
- SSO configuration UI for tenants
- JIT user provisioning logic
- ≥10 integration tests

57 User-Generated API Keys

Developer API

SPECS

- Models: api_keys (id, prefix, hash, user_id, permissions_json, last_used, expires_at). Never store plain secret.
- Auth: Bearer {api_key}. Middleware validates hash and scopes.
- UI: Developer Settings panel. 'Generate New Key' modal. List active keys. Revoke button.

OUTPUTS

- API Key generation and hashing service
- Scope-based authorization middleware
- Developer Settings UI
- ≥12 API tests, ≥3 E2E tests

58

Outbound Webhooks

Integration

SPECS

- Dispatcher: Background worker listening to event bus. Matches events to subscriptions.
- Models: webhook_subscriptions (id, user_id, url, event_types, secret).
- Delivery: POST payload with HMAC-SHA256 signature header. Exponential backoff for failures.

OUTPUTS

- Webhook dispatch service
- Subscription management API
- Webhook UI in Developer Settings
- Signature verification docs
- ≥10 integration tests

59

Mobile App Foundation (React Native)

Mobile

SPECS

- Stack: React Native, Expo, React Navigation, NativeWind (Tailwind).
- Auth: SecureStore for storing JWT. Biometric prompt on launch.
- Architecture: Share business logic types/utilities with web frontend via monorepo or git submodule.

OUTPUTS

- React Native project structure
- CI/CD for mobile builds (TestFlight/APK)
- Authentication screen and logic
- Shared types library

60

Mobile Authentication & Biometrics

Mobile Security

SPECS

- Biometrics: Use expo-local-authentication. Fallback to PIN if biometrics unavailable.
- Sessions: Register device_id / user_agent on login. API: /api/auth/devices.
- Push: Firebase Cloud Messaging (FCM) setup. Store device_token on login.

OUTPUTS

- Biometric unlock screen
- Device management API and UI
- FCM token registration
- ≥5 mobile integration tests

61 Mobile Watchlist & Charts

Mobile Feature

SPECS

- Components: Native implementation of Watchlist (FlatList). Charts: WebView pointing to lightweight charts (easiest) or native rewrite.
- Optimization: Throttle WS updates on mobile to save battery.
- Gestures: Swipe to delete watchlist item. Pinch-to-zoom on chart.

OUTPUTS

- Mobile Watchlist screen
- Mobile Chart screen
- Real-time data hook for mobile
- Gestures implementation

62 Notification Center (Unified)

Engagement

SPECS

- Backend: notification_preferences table. Dispatcher checks preferences before sending.
- Mobile: Handle background/foreground push notifications. Deep link to relevant screen (e.g., tap Alert → open Chart).
- Web: Notification dropdown with 'Mark all read'.

OUTPUTS

- Unified Notification Center (Web + Mobile)
- Push notification dispatch (FCM)
- Preference settings UI
- Deep linking logic

63 ChatBots (Slack/Discord/Telegram)

Integration

SPECS

- Architecture: BotGateway service. Webhook receivers for each platform.
- Mapping: Link Slack User ID to Platform User ID (Oauth flow).
- Commands: Text parser or slash command handlers. 'Subscribe' to channel updates.

OUTPUTS

- Slack/Discord/Telegram bot implementations
- BotGateway service
- User linking UI (Oauth)
- Command parser

64 Zapier & IFTTT Integration

Ecosystem

SPECS

- API: Expose Swagger/OpenAPI definition tailored for Zapier.
- Auth: OAuth2 implementation for Zapier connection.
- Triggers: REST Hooks (Zapier subscribes to our webhooks).

OUTPUTS

- Zapier App CLI project
- OAuth2 provider endpoints
- Zapier triggers/actions definitions
- Documentation for no-code users

65

Q1 Review & Security Audit

Review

SPECS

- Audit Scope: OWASP Top 10 on new endpoints. IDOR checks on multi-tenant data. Token leakage checks.
- Load Test: Mobile API endpoints under high concurrency.
- Cleanup: Deprecate v1 endpoints if any.

OUTPUTS

- Security Audit Report
- Q1 Retrospective
- Year 2 Q2 Roadmap
- Performance Baseline

66 Horizontal Scaling (Kubernetes)

Infrastructure

SPECS

- K8s: Deployments for API, Workers, WebSocket Hub. StatefulSets for DB (if self-hosted) or connect to managed DB.
- HPA: Horizontal Pod Autoscaler targeting 70% CPU.
- Ingress: Nginx Ingress Controller for load balancing.

OUTPUTS

- Helm charts for all services
- K8s cluster configuration (terraform)
- CI/CD pipeline updating image tags
- ≥3 Load tests

67 Database Sharding & Partitioning

Database

SPECS

- Partitioning: TimescaleDB (Postgres extension) for time-series data. Partition by time (weekly chunks).
- Sharding: Tenant-based sharding (Schema per tenant vs Row-based). Choose Schema-based for isolation ease.
- Archival: Move chunks >1 year old to S3/Glacier compressed (Parquet).

OUTPUTS

- TimescaleDB migration
- Data retention policy application
- Sharding logic (middleware routing)
- ≥5 performance benchmarks

68 Global CDN & Edge Caching

Latency

SPECS

- CDN: Cache static JS/CSS, images, and public market data API responses (with short TTL).
- Edge: Validate JWTs at the edge to block unauthorized requests before they hit origin.
- Geo-routing: Route users to nearest backend region (if multi-region).

OUTPUTS

- CDN configuration (Terraform)
- Edge Worker scripts (Auth)
- Cache invalidation strategy
- ≥3 latency tests

69 High-Performance Data Ingestion (Rust)

Optimization

SPECS

- Rust Service: Connects to exchange WS, parses JSON/Binary, normalizes, publishes to Redpanda/Kafka.
- Performance: Zero-copy parsing (serde_json/simd-json). Lock-free ring buffers.
- Integration: Python backend consumes from Kafka/Redpanda.

OUTPUTS

- Rust ingestion microservice
- Kafka/Redpanda integration
- Benchmark report
- Migration plan

70

Order Execution Latency Optimization

Optimization

SPECS

- Profiling: Distributed tracing (Jaeger) with microsecond precision.
- Optimization: Kernel bypass (unlikely for web app, but maybe for engine), removing DB writes from critical path (async persist).
- Co-location: Deploy execution engine in AWS/GCP region closest to exchange.

OUTPUTS

- Latency profiling report
- Optimized execution engine
- Async persistence architecture
- ≥5 latency benchmarks

71 Historical Data Compression (Parquet)

Data Efficiency

SPECS

- Format: Parquet with Snappy compression. Partition by symbol/year.
- Query Engine: DuckDB embedded in the Python backend for interactive analytics on Parquet files.
- Pipeline: Daily job converts yesterday's DB data to Parquet in S3.

OUTPUTS

- Data conversion pipeline (Airflow/Cron)
- DuckDB query integration
- S3 Data Lake architecture
- ≥5 query performance tests

72

AI Model Serving Optimization

AI Infrastructure

SPECS

- Format: Convert PyTorch/TensorFlow models to ONNX.
- Serving: Triton Inference Server or simple ONNX Runtime in Python process.
- Optimization: Quantization (INT8) if accuracy loss is acceptable (<1%).

OUTPUTS

- ONNX conversion pipeline
- Optimized inference service
- Model versioning system
- ≥3 benchmark reports

73

Load Testing at Scale

Reliability

SPECS

- Tool: K6 or Locust distributed load generation.
- Scenarios: Login storm, WebSocket broadcast fan-out, heavy backtest requests.
- Monitoring: Grafana/Prometheus dashboard watching all services.

OUTPUTS

- K6/Locust load test scripts
- Load test report with bottlenecks
- Tuning configuration (sysctl, ulimits)
- ≥3 full scale tests

74

Rate Limiting & DDoS Protection

Security

SPECS

- Rate Limit: Redis-backed sliding window. 100/min for API, 5/min for heavy jobs.
- WAF: Cloudflare or AWS WAF. Block malicious User-Agents, SQLi patterns.
- Circuit Breakers: Stop calling downstream services (exchanges) if they error out.

OUTPUTS

- Redis rate limiter middleware
- WAF configuration
- Circuit breaker implementation
- ≥8 unit tests, ≥2 integration tests

75

Infrastructure as Code (IaC)

DevOps

SPECS

- Terraform: Modules for VPC, K8s, RDS, Redis, S3, IAM.
- State Management: S3 backend with locking.
- Environments: Dev, Staging, Prod defined via variabels.

OUTPUTS

- Complete Terraform repository
- DR plan documentation
- Automated environment provisioning pipeline
- ≥1 disaster recovery drill

76 Cost Optimization

Ops

SPECS

- Spot: Use K8s Spot interaction handling. 60-80% savings.
- Auto-scaling: aggressively scale down in off-hours.
- Storage: Lifecycle policies for S3 logs/artifacts.

OUTPUTS

- Cost optimization report
- Spot instance configuration
- Auto-scaling schedule
- Budget alerts

77 Chaos Engineering

Resilience

SPECS

- Tool: Chaos Mesh or Gremlin in K8s.
- Experiments: Pod Chaos (random kill), Network Chaos (latency/packet loss), Stress Chaos (CPU burn).
- Game Day: Scheduled team exercise to respond to chaos.

OUTPUTS

- Chaos engineering suite
- Resilience report
- Runbooks for incident response
- ≥3 Game Days conducted

78

Q2 Review & Performance Certification

Review

SPECS

- Certification: 'Apex Certified Scalable' badge if all KPIs met (10k users, <50ms latency, 99.99% uptime).
- Metrics Review: latency, throughput, error rate, cost.
- Tech Debt: Clean up any hacks from optimization phase.

OUTPUTS

- Q2 Performance Report
- Scalability Certification
- Year 2 Q3 Roadmap

WAVE 6 // WEEKS 79-104

INTELLIGENT AGENTS

PRIMARY OBJECTIVES

- LLM Agent Builders
- Workflow Generation
- Natural Language Ops

KEY DELIVERABLES

- Agent Sandbox
- NL Query Engine
- Workflow Studio

DEFINITION OF DONE // REALITY CHECK

- [x] Telemetry Emitted & Logged
- [x] Search Index Updated
- [x] Platform Health Monitor Green
- [x] Export Artifact Bundle Generated
- [x] Playwright MCP Headed Proof
- [ ] Video Walkthrough Recorded

79

Reinforcement Learning (RL) Framework

AI Innovation

SPECS

- Env: Custom Gym environment wrapping the backtester. Observation: OHLCV + Indicators. Action: Buy/Sell/Hold/Size.
- Library: Stable Baselines3 or Ray RLLib.
- Training: Background job running episodes. Checkpointing models.

OUTPUTS

- TradingGymEnvironment class
- RL training pipeline
- Model checkpoint manager
- Learning curve visualization

80

Custom AI Agent Builder (No-Code)

User Tools

SPECS

- UI: React Flow or Rete.js. Drag-and-drop nodes.
- Compiler: Traverse graph → generate Python class inheriting from StrategyBase.
- Execution: Run generated code in backtester or live engine.

OUTPUTS

- AgentBuilder UI canvas
- Graph-to-Code compiler
- Node library (20+ nodes)
- ≥10 graph compilation tests

81 Generative AI for Strategy Research

GenAI

SPECS

- LLM Client: System prompt with Strategy API documentation context.
- Interface: Chat panel in Strategy Editor. 'Insert Code' button.
- Safety: Static analysis (AST) on generated code to prevent malicious imports.

OUTPUTS

- LLM integration service
- Strategy generation prompt templates
- Code safety validator
- Chat interface

82 Sentiment Analysis 2.0 (LLM-based)

NLP

SPECS

- Pipeline: News API → Vector DB (optional) → LLM (batch processing) → Sentiment Score.
- Entity Extraction: Identify ticker symbols mentioned.
- Dashboard: Real-time sentiment heat map ticker.

OUTPUTS

- LLM Sentiment Analysis pipeline
- Entity extraction module
- News impact scorer
- Sentiment dashboard widget

83 Predictive Market Regimes

ML

SPECS

- Model: GaussianHMM unsupervised learning on returns/volatility.
- Signal: current_regime (0=LowVol, 1=HighVol, 2=Crash).
- UI: Color-coded background regions on price chart.

OUTPUTS

- Market Regime HMM model
- Regime detection service
- Chart background visualization
- ≥3 strategy examples using regimes

84 Agent Collaboration (Swarm)

Multi-Agent

SPECS

- Communication: Shared Blackboard or direct Message Passing.
- Coordination: Voting mechanism (e.g., 2/3 agents must agree to trade).
- Config: Swarm composition UI.

OUTPUTS

- Agent messaging infrastructure
- Voting/Consensus logic
- Swarm configuration UI
- ≥5 Interaction tests

85

Explainable AI (XAI) Dashboard

Trust

SPECS

- XAI Lib: SHAP (SHapley Additive exPlanations).
- Integration: On trade signal, calculate SHAP values for top 5 features.
- UI: Feature contribution bar chart for every ML-driven trade.

OUTPUTS

- SHAP integration
- Feature importance calculation pipeline
- Trade explanation UI
- ≥5 XAI correctness tests

86 AI-Driven Risk Management

Risk

SPECS

- Model: VaR forecasting using LSTM/Transformer.
- Sizing: size = base_size * model_confidence.
- Guardian: AI watchdog monitors all trades and kills positions behaving anomalously.

OUTPUTS

- Predictive Risk Model
- Dynamic sizing logic
- AI Guardian service
- ≥5 Backtests

87

Conversational Interface (Chat with Data)

UX

SPECS

- LLM Agent: LangChain SQLDatabaseToolkit.
- Schema Info: Provide sanitized schema to LLM.
- Visuals: LLM generates Plotly JSON configuration for unexpected queries.

OUTPUTS

- Conversational Agent backend
- Chat UI with chart rendering support
- Sanitized schema definition
- ≥20 NLQ test cases

88 Automated Feature Engineering

AutoML

SPECS

- Library: tsfresh or custom genetic programming.
- Pipeline: Raw Data → Feature Generation → Selection → Model Training.
- UI: Feature importance ranking table.

OUTPUTS

- Auto-feature engineering pipeline
- Feature selection module
- Integration with Strategy Builder

89

Walk-Forward Optimization (AI-Tuned)

Optimization

SPECS

- Engine: Split data into Train/Test chunks. Train on window N, Test on N+1. Slide window.
- Optimizer: Optuna with TPE/Gaussian Process.
- Report: Out-of-sample equity curve vs In-sample.

OUTPUTS

- Walk-Forward engine
- Bayesian Optimizer (Optuna)
- WFO Report component
- ≥5 validation tests

90

AI Model Marketplace

Marketplace

SPECS

- Registry: models table with metadata, metrics, and price.
- Security: Models executed in secure sandbox. Creator cannot see user data; User cannot see model weights.
- Licensing: Pay-per-use or Subscription.

OUTPUTS

- Model Marketplace UI
- Secure Execution Environment (black box)
- Licensing/Payment integration stub

91

Q3 Review & AI Governance

Review

SPECS

- Governance: AI Factsheets for every model (Training data, limitations, bias).
- Monitoring: Drift detection for all production models.
- Retrospective: Assess value add of AI features.

OUTPUTS

- AI Governance Policy
- Model Monitoring Dashboard
- Q3 Retrospective Report
- Year 2 Q4 Roadmap

92 Alternative Data Pipeline

Data Expansion

SPECS

- Sources: Orbital Insight, SimilarWeb API. Ingest to Data Lake (Parquet).
- Correlation: Rolling correlation between Alt Data Signal and Price Return.
- UI: Alt Data workspace with heatmap visualization.

OUTPUTS

- Alt Data Ingestion Pipeline
- Correlation Engine
- Alt Data UI Visualization

93

Causality Detection Engine

Advanced Stats

SPECS

- Library: CausalImpact or custom Granger Causality test implementation.
- Analysis: Run pairwise causality tests on all 500+ features.
- Report: Directed Acyclic Graph (DAG) of market drivers.

OUTPUTS

- Causality Detection Service
- Market Driver DAG Visualization
- Causal Signal Filter

94 Quantitative Strategy Portfolio

Strategy

SPECS

- Portfolio: Collection of 10+ uncorrelated strategies.
- Optimizer: Quadratic Programming solver (cvxpy) to minimize variance for target return.
- Execution: Rebalancing algorithm generates target orders.

OUTPUTS

- Strategy Portfolio Manager
- Portfolio Optimizer (cvxpy)
- Automated Rebalancer

95

High-Frequency Data Analytics

Big Data

SPECS

- Data: Tick-level L3 data. Store in KDB+ or heavily optimized Parquet.
- Metrics: OBI, VWAP, TWAP, Order Flow Toxicity (VPIN).
- UI: Depth Chart 2.0 with historical liquidity heat map.

OUTPUTS

- L3 Data Analytics Engine
- Microstructure Metrics Library
- Liquidity Heatmap UI

96 Automated Risk Hedging

Risk

SPECS

- Hedge Logic: Calculate Portfolio Beta. If > Target, short S&P500 futures to neutralize.
- Options: Buy OTM puts if VIX spikes.
- Execution: Hedge orders routed to distinct 'Hedge' account.

OUTPUTS

- Auto-Hedging Service
- Beta/Delta Monitor
- Hedge Execution Algo

97 Institutional Reporting Engine

Enterprise

SPECS

- Generator: WeasyPrint or ReportLab (Python). Templates in Jinja2.
- Metrics: Active Return, Information Ratio, Tracking Error.
- Distribution: Email report to investors automatically.

OUTPUTS

- PDF Report Generator
- Attribution Analysis Module
- Email Distribution System

98

Privacy-Preserving AI (Federated Learning)

Privacy

SPECS

- Framework: TensorFlow Federated or PySyft.
- Toplogy: Central server aggregates gradients, not data.
- Privacy: Add noise to gradients (Differential Privacy).

OUTPUTS

- Federated Learning Prototype
- Privacy Guarantee Whitepaper
- Global Model Training Pipeline

99

Quantum-Resistant Cryptography

Security

SPECS

- Library: liboqs (Open Quantum Safe) via Python wrapper.
- Migration: Dual-signing (Classical + Post-Quantum) during transition.
- Storage: Re-encrypt secrets with PQ algorithms.

OUTPUTS

- PQ Crypto Library Integration
- Key Migration Plan
- Dual-Signing Implementation

100 Global Market Connectivity

Expansion

SPECS

- Connectors: FIX protocol adapters for international exchanges.
- Time: strict UTC internal, localized presentation.
- Currency: Real-time FX conversion for unified purchasing power.

OUTPUTS

- Global Exchange Connectors (FIX)
- Multi-currency Margin Engine
- Timezone-aware scheduler

101 Decentralized Data Marketplace

Web3

SPECS

- Blockchain: Polygon or L2. Smart Contract: DataToken (ERC-20/721).
- Storage: Encrypted chunks on IPFS.
- Access: 'Unlock' data via crypto payment.

OUTPUTS

- Data Marketplace Smart Contracts
- Web3 Wallet Integration
- IPFS Storage Bridge

102 Final Polish & Release Prep

Release

SPECS

- Audit: extensive manual testing of every feature.
- Documentation: Sphinx/MkDocs update. Video tutorials.
- Release branching: v2.0.0-rc1.

OUTPUTS

- v2.0 Release Candidate
- Updated Documentation Suite
- Release Notes

103 Year 2 Review & Roadmap

Review

SPECS

- Analytics: Compile 2-year growth stats.
- Team: Performance review.
- Vision: Brainstorming sessions.

OUTPUTS

- Year 2 Retrospective
- Year 3 Vision Paper

104 Launch Party

Celebration

SPECS

- Event: Livestream setup.
- Deployment: Flip the switch on v2.0 prod.

OUTPUTS

- Useable v2.0 Platform
- Launch Event

VISION TRACK

YEARS 3 & 4 // R&D & SINGULARITY

YEAR 3: GLOBAL DOMINANCE

SPECULATIVE RESEARCH DIRECTION

- **W105**: Planetary Scale Data
- **W106**: Security Singularity
- **W107**: Alternative Data
- **W108**: Autonomous Expansion
- **W109**: Autonomous Expansion
- **W110**: Autonomous Expansion
- **W111**: Autonomous Expansion
- **W112**: Autonomous Expansion
- **W113**: Autonomous Expansion
- **W114**: Autonomous Expansion

YEAR 4: SINGULARITY

SPECULATIVE RESEARCH DIRECTION

W157: Inter-Planetary Arbitrage

W158: Inter-Planetary Arbitrage

W159: Inter-Planetary Arbitrage

W160: Inter-Planetary Arbitrage

W161: Inter-Planetary Arbitrage

W162: Inter-Planetary Arbitrage

W163: Inter-Planetary Arbitrage

W164: Inter-Planetary Arbitrage

W165: Inter-Planetary Arbitrage

W166: Inter-Planetary Arbitrage